

CUDA

Parallel programming on NVIDIA GPUs

What is CUDA?

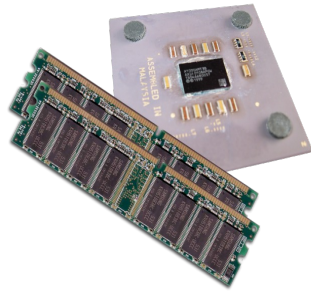
- CUDA Architecture
 - Expose GPU parallelism for scientific computing
 - Provide high performance for scientific computing
- CUDA C/C++ and CUDA Fortran
 - Based on industry-standard C/C++ and Fortran
 - Small set of extensions to enable heterogeneous programming
 - Straightforward APIs to manage devices, memory etc.

Introduction to CUDA C/C++

- What will you learn in this session?
 - Start from “Hello World!”
 - Write and launch CUDA C/C++ kernels
 - Manage GPU memory
 - Manage communication and synchronization

Terminology

- *Host* The CPU and its memory (host memory)
- *Device* The GPU and its memory (device memory)



Host



Device

Heterogeneous Computing

```
#include <cuda.h>
#include <stdio.h>

#define N 100000
#define M 256

__global__ void add(int *a, int *b, int *c) {
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    c[index] = a[index] + b[index];
}

void random_ints(int* x, int size){
    int i;
    for (i=0; i<size; i++) x[i]=rand()%10;
}

int main(void) {
    int *a, *b, *c;          // host copies of a, b, c
    int *a_d, *b_d, *c_d;   // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&a_d, size);
    cudaMalloc((void **)&b_d, size);
    cudaMalloc((void **)&c_d, size);

    // Alloc space for host copies of a, b, c and setup
    // input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // Copy inputs to device
    cudaMemcpy(a_d, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(b_d, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU
    add<<<(N+M-1)/M,M>>>(a_d, b_d, c_d);

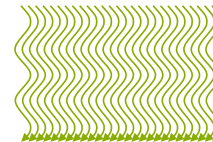
    // Copy result back to host
    cudaMemcpy(c, c_d, size, cudaMemcpyDeviceToHost);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(a_d); cudaFree(b_d); cudaFree(c_d);
    return 0;
}
```

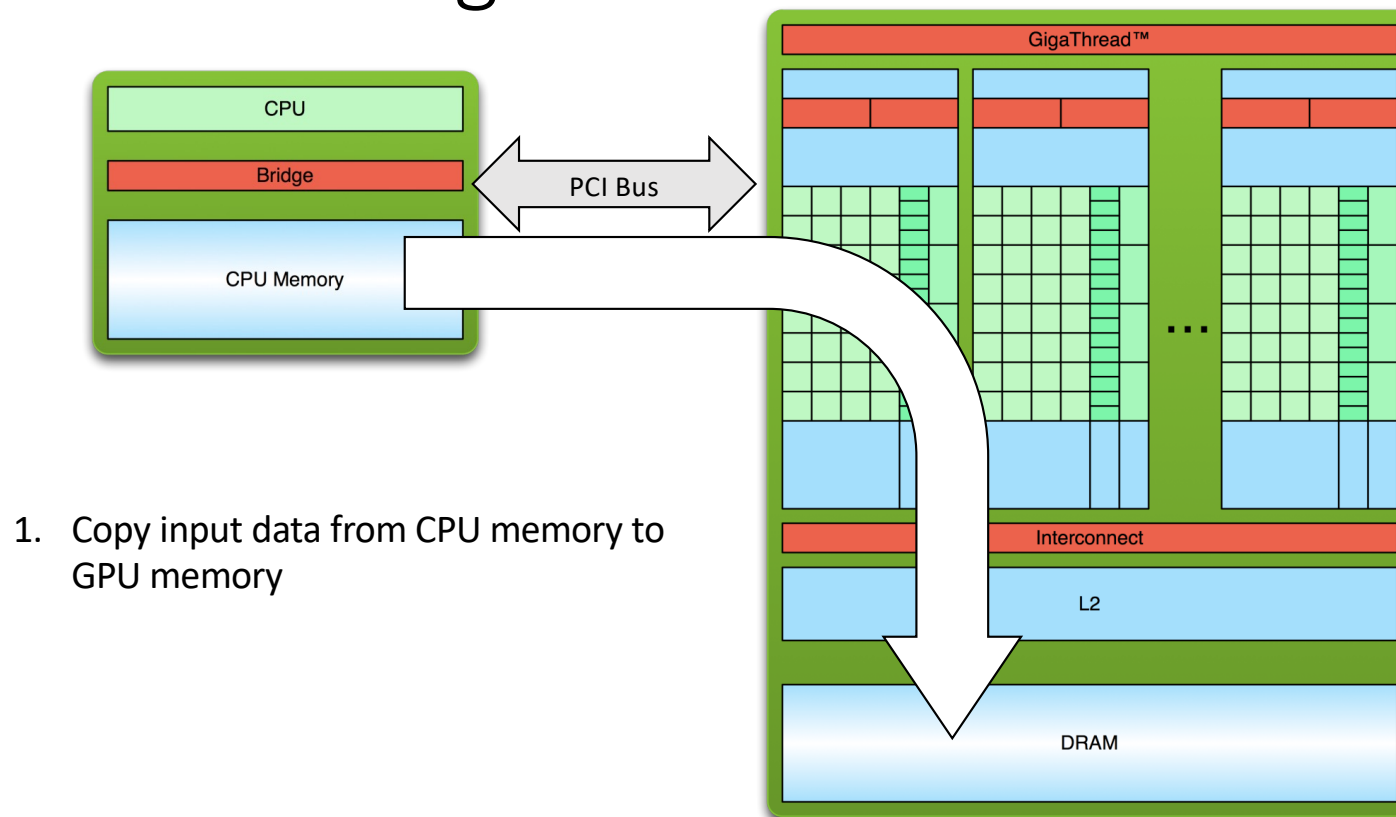
} parallel function

} serial code

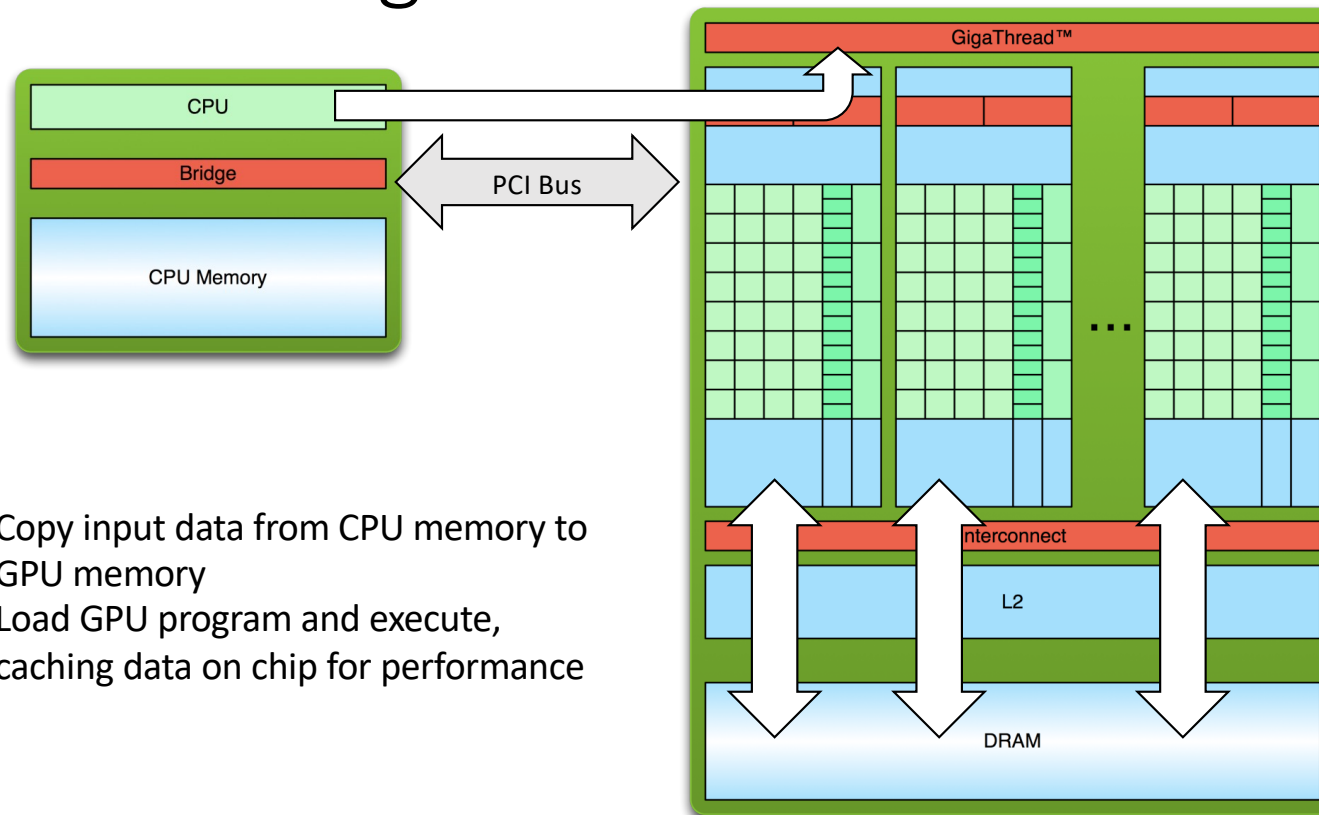
} parallel code



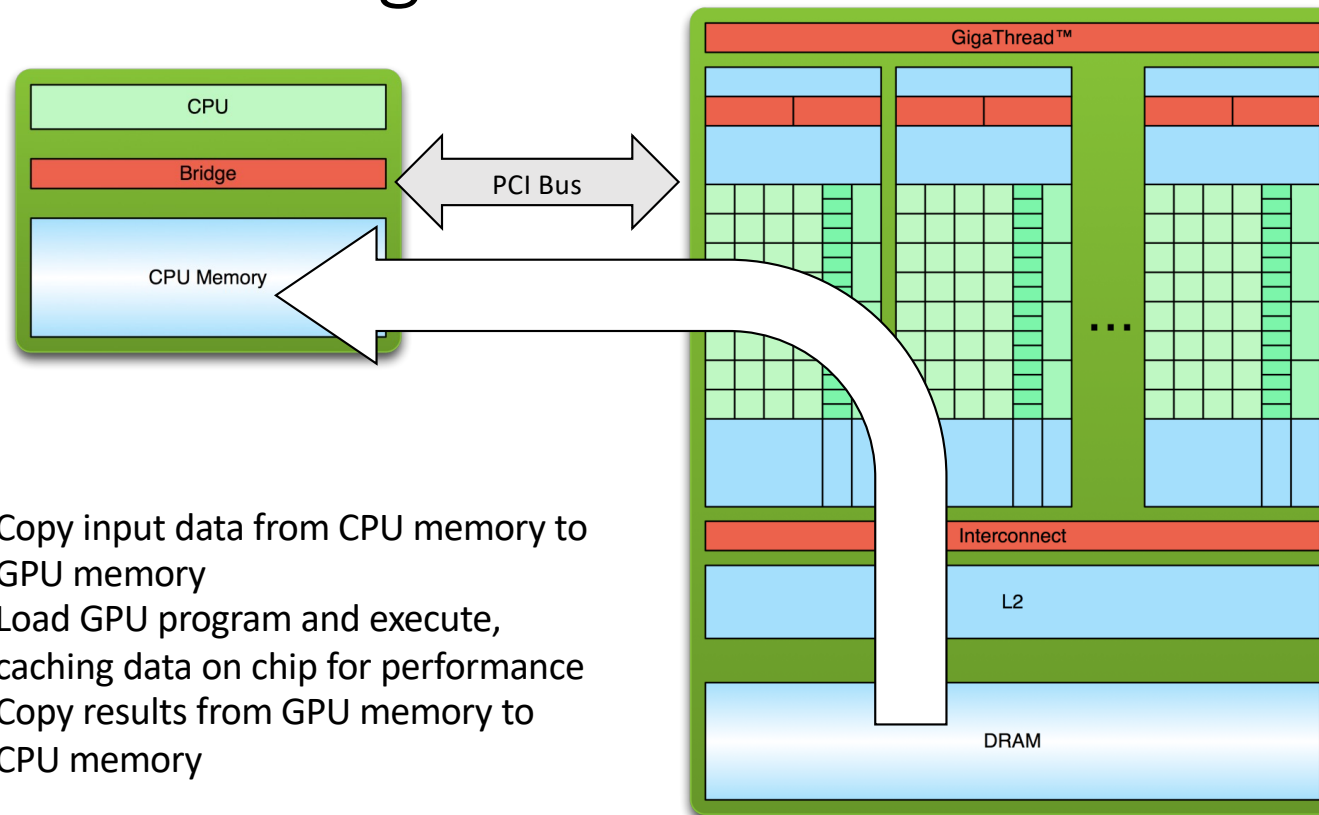
Simple Processing Flow



Simple Processing Flow



Simple Processing Flow



Hello World!

```
int main(void) {  
    printf("Hello World!\n");  
    return 0;  
}
```

```
$ nvcc hello_world.cu Compilation  
$ ./a.out Execute  
Hello World! Output
```

Standard C or Fortran that runs on the **host**.

NVIDIA compilers (nvcc or nvfortran) can also be used to compile programs with *no device code*.

Hello World! with Device Code

```
__global__ void mykernel(void) {  
}
```

Device

```
int main(void) {  
    mykernel<<<1,1>>>();  
    printf("Hello World!\n");  
    return 0;  
}
```

Host

- Two new syntactic elements...

Hello World! with Device Code

```
__global__ void mykernel(void) {}
```

- The `__global__` keyword indicates a function or kernel that:
 - Runs on the **device**
 - Is called from the **host**
- **nvcc** separates source code into **host** and **device** components
 - **Device** functions (e.g. `mykernel()`) processed by NVIDIA compiler
 - **Host** functions (e.g. `main()`) processed by standard **host** compiler **gcc**.

Hello World! with Device Code

```
mykernel<<<1,1>>>();
```

- Triple angle brackets mark a call from *host* code to *device* code
 - Also called a “kernel launch”
 - Where the arguments inside correspond to (*blocks*, *threads_per_block*), we will return to this terminology later.
- That’s all that is required to execute a function on the GPU!

Hello World! with Device Code

```
__global__ void mykernel(void) {  
}
```

```
int main(void) {  
    mykernel<<<1,1>>>();  
    printf("Hello World!\n");  
    return 0;  
}
```

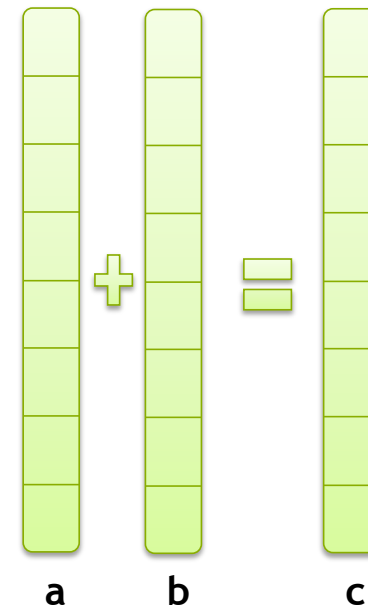
Result:

```
$ nvcc hello.cu  
$ ./a.out  
Hello World!
```

- So far **mykernel()** does nothing, let's write a parallel code...

Parallel Programming in CUDA

- GPU computing is about massive parallelism
- Let's start by adding two integers and build up to adding two vectors



Addition on the Device

- A simple kernel to add two integers

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- As before `__global__` is a CUDA keyword meaning
 - `add()` will be executed on the **device**
 - `add()` will be called from the **host**

Addition on the Device

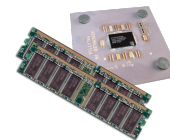
- Note that we use pointers for the variables

```
__global__ void add(int *a, int *b, int *c) {  
    *c = *a + *b;  
}
```

- `add()` runs on the **device**, so `a`, `b` and `c` must point to **device** memory
- In other words, we need to allocate memory on the GPU

Memory Management

- Host and device memory are separate entities
 - *Device* pointers point to GPU memory
 - May be passed to/from *host* code
 - May *not* be dereferenced in *host* code
 - *Host* pointers point to CPU memory
 - May be passed to/from *device* code
 - May *not* be dereferenced in *device* code
- Simple CUDA API for handling device memory
 - `cudaMalloc()`, `cudaFree()`, `cudaMemcpy()`
 - Similar to the C equivalents `malloc()`, `free()`, `memcpy()`



Addition on the Device: `add()`

- Returning to our `add()` kernel

```
__global__ void add(int *a, int *b, int *c)    {  
    *c = *a + *b;  
}
```

- Let's take a look at `main()`...

Addition on the Device: `main()`

```
int main(void) {  
    int a, b, c;                // host copies of a, b, c  
    int *a_d, *b_d, *c_d;      // device copies of a, b, c  
    int size = sizeof(int);  
  
    // Allocate space for device copies of a, b, c  
    cudaMalloc((void **)&a_d, size);  
    cudaMalloc((void **)&b_d, size);  
    cudaMalloc((void **)&c_d, size);  
  
    // Setup input values  
    a = 2;  
    b = 7;
```

Addition on the Device: `main()`

```
// Copy inputs to device
cudaMemcpy(a_d, &a, size, cudaMemcpyHostToDevice);
cudaMemcpy(b_d, &b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU
add<<<1,1>>>>(a_d, b_d, c_d);

// Copy result back to host
cudaMemcpy(&c, c_d, size, cudaMemcpyDeviceToHost);

// Print the result
printf('c = %d', c);

// Cleanup
cudaFree(a_d); cudaFree(b_d); cudaFree(c_d);
return 0;
}
```

Moving to Parallel

- GPU computing is about massive parallelism
 - So how do we run code in parallel on the device?

```
add<<< 1, 1 >>>() ;
```



```
add<<< N, 1 >>>() ;
```

- Instead of executing `add()` once, execute `N` times in parallel

Vector Addition on the Device

- With `add()` running in parallel we can do vector addition
- Terminology: each parallel invocation of `add()` is referred to as a `block`
 - The set of `blocks` is referred to as a `grid`
 - Each invocation can refer to its `block` index using `blockIdx.x`

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- By using `blockIdx.x` to index into the array, each block handles a different index

Vector Addition on the Device

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- On the **device**, each **block** can be executed in parallel:

Block 0

```
c[0] = a[0] + b[0];
```

Block 1

```
c[1] = a[1] + b[1];
```

Block 2

```
c[2] = a[2] + b[2];
```

Block 3

```
c[3] = a[3] + b[3];
```

Vector Addition on the Device: `add()`

- Returning to our parallelized `add()` kernel

```
__global__ void add(int *a, int *b, int *c) {  
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];  
}
```

- Let's take a look at `main()`...

Vector Addition on the Device:

`main()`

```
#define N 512
int main(void) {
    int *a, *b, *c          // host copies of a, b, c
    int *a_d, *b_d, *c_d;  // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&a_d, size);
    cudaMalloc((void **)&b_d, size);
    cudaMalloc((void **)&c_d, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Vector Addition on the Device:

main()

// Copy inputs to device

cudaMemcpy(a_d, a, size, cudaMemcpyHostToDevice);

cudaMemcpy(b_d, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU with N blocks

add<<<N,1>>>(a_d, b_d, c_d);

// Copy result back to host

cudaMemcpy(c, c_d, size, cudaMemcpyDeviceToHost);

// Cleanup

free(a); free(b); free(c);

cudaFree(a_d); cudaFree(b_d); cudaFree(c_d);

return 0;

}

Recapitulation

- Difference between *host* and *device*
 - *Host* CPU
 - *Device* GPU
- Using `__global__` to declare a function as *device* code
 - Executes on the *device*
 - Called from the *host*
- Passing parameters from *host* code to a *device* function

Recapitulation

- Basic device memory management
 - `cudaMalloc()`
 - `cudaMemcpy()`
 - `cudaFree()`
- Launching parallel kernels
 - Launch `N` copies of `add()` with `add<<<N,1>>>(...)` ;
 - Use `blockIdx.x` to access `block` index

CUDA Threads

- Terminology: a **block** can be split into parallel **threads**
- Let's change `add()` to use parallel **threads** instead of parallel **blocks**

```
__global__ void add(int *a, int *b, int *c) {  
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];  
}
```

- We use **threadIdx.x** instead of **blockIdx.x**
- Need to make one change in `main()` ...

Vector Addition Using Threads:

main()

```
#define N 512
int main(void) {
    int *a, *b, *c;           // host copies of a, b, c
    int *a_d, *b_d, *c_d;     // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&a_d, size);
    cudaMalloc((void **)&b_d, size);
    cudaMalloc((void **)&c_d, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Vector Addition Using Threads:

main()

```
// Copy inputs to device
cudaMemcpy(a_d, a, size, cudaMemcpyHostToDevice);
cudaMemcpy(b_d, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU with N threads
add<<<1,N>>>>(a_d, b_d, c_d);

// Copy result back to host
cudaMemcpy(c, c_d, size, cudaMemcpyDeviceToHost);

// Cleanup
free(a); free(b); free(c);
cudaFree(a_d); cudaFree(b_d); cudaFree(c_d);
return 0;
}
```

Combining Blocks and Threads

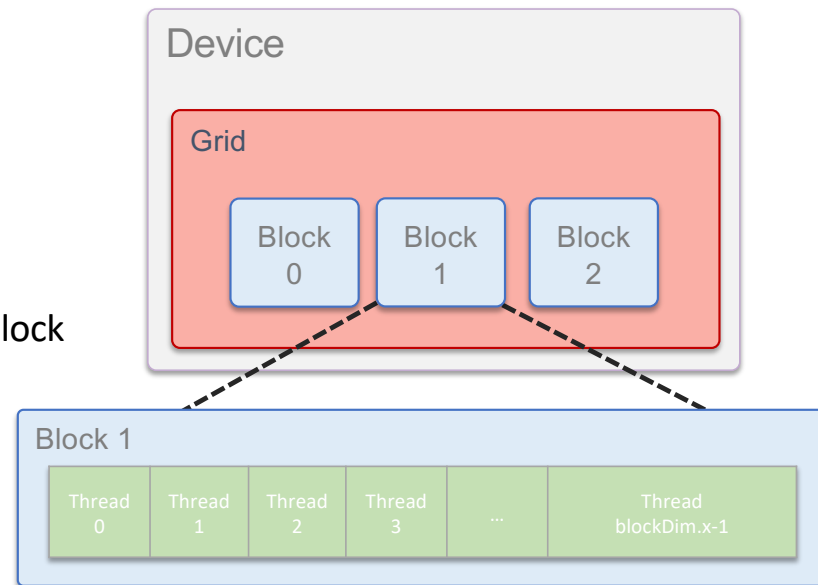
- We've seen parallel vector addition using:
 - Many blocks with one thread each `add<<<N, 1>>>`
 - One block with many threads `add<<<1, N>>>`
- Let's adapt vector addition to use the full parallel power of the Device by incorporating both blocks and threads
- Let's start with data indexing...

Indexing

- A kernel is launched as a **grid** of **blocks** of **threads**

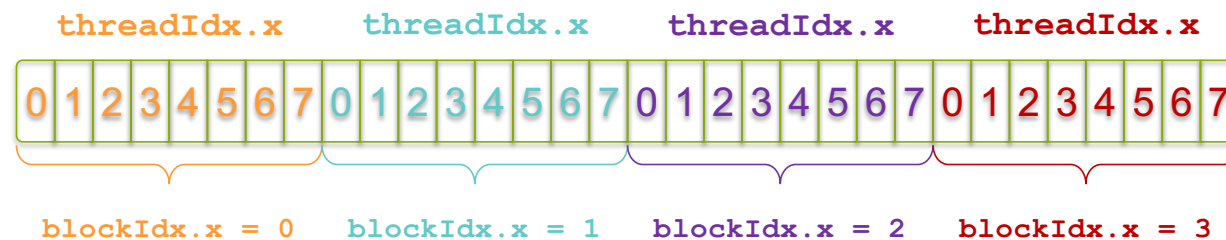
Built-in variables:

threadIdx.x Thread index inside the Block
blockIdx.x Block index
blockDim.x Threads per Block
gridDim.x Blocks



Indexing Arrays

- No longer as simple as using `blockIdx.x` and `threadIdx.x`
 - Consider indexing an array with one element per thread (8 threads/block)

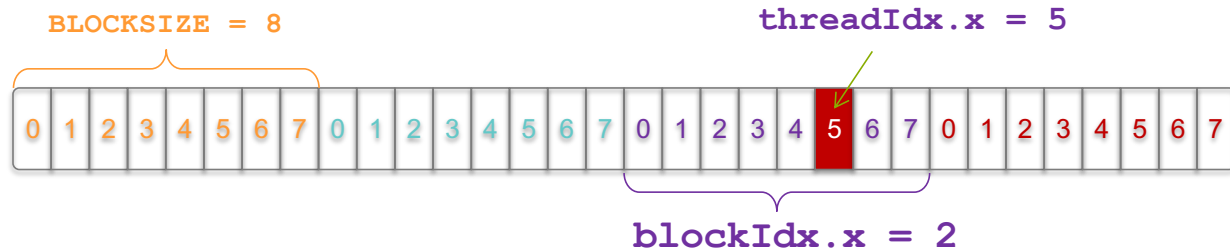


- With 8 threads/block a unique index for each thread is given by:

```
int index = threadIdx.x + blockIdx.x * 8;
```

Indexing Arrays: Example

- Which thread will operate on the red element?



```
int index = threadIdx.x + blockIdx.x * BLOCKSIZE;  
          =           5      +           2      * 8;  
          = 21;
```

Vector Addition with Blocks and Threads

- Use the built-in variable `blockDim.x` for threads per block

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
```

- Combined version of `add()` to use *parallel threads and parallel blocks*

```
__global__ void add(int *a, int *b, int *c) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    c[index] = a[index] + b[index];  
}
```

- What changes need to be made in `main()`?

Addition with Blocks and Threads:

main()

```
#define N (2048*2048)
#define BLOCKSIZE 512
int main(void) {
    int *a, *b, *c;           // host copies of a, b, c
    int *a_d, *b_d, *c_d;     // device copies of a, b, c
    int size = N * sizeof(int);

    // Alloc space for device copies of a, b, c
    cudaMalloc((void **)&a_d, size);
    cudaMalloc((void **)&b_d, size);
    cudaMalloc((void **)&c_d, size);

    // Alloc space for host copies of a, b, c and setup input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);
```

Addition with Blocks and Threads:

main()

// Copy inputs to device

cudaMemcpy(a_d, a, size, cudaMemcpyHostToDevice);

cudaMemcpy(b_d, b, size, cudaMemcpyHostToDevice);

// Launch add() kernel on GPU

add<<<N/BLOCKSIZE,BLOCKSIZE>>>(a_d, b_d, c_d);

// Copy result back to host

cudaMemcpy(c, c_d, size, cudaMemcpyDeviceToHost);

// Cleanup

free(a); free(b); free(c);

cudaFree(a_d); cudaFree(b_d); cudaFree(c_d);

return 0;

}

Handling Arbitrary Vector Sizes

- Typical problems are not friendly multiples of `blockDim.x`
- Avoid accessing beyond the end of the arrays:

```
__global__ void add(int *a, int *b, int *c, int n) {  
    int index = threadIdx.x + blockIdx.x * blockDim.x;  
    if (index < n)  
        c[index] = a[index] + b[index];  
}
```

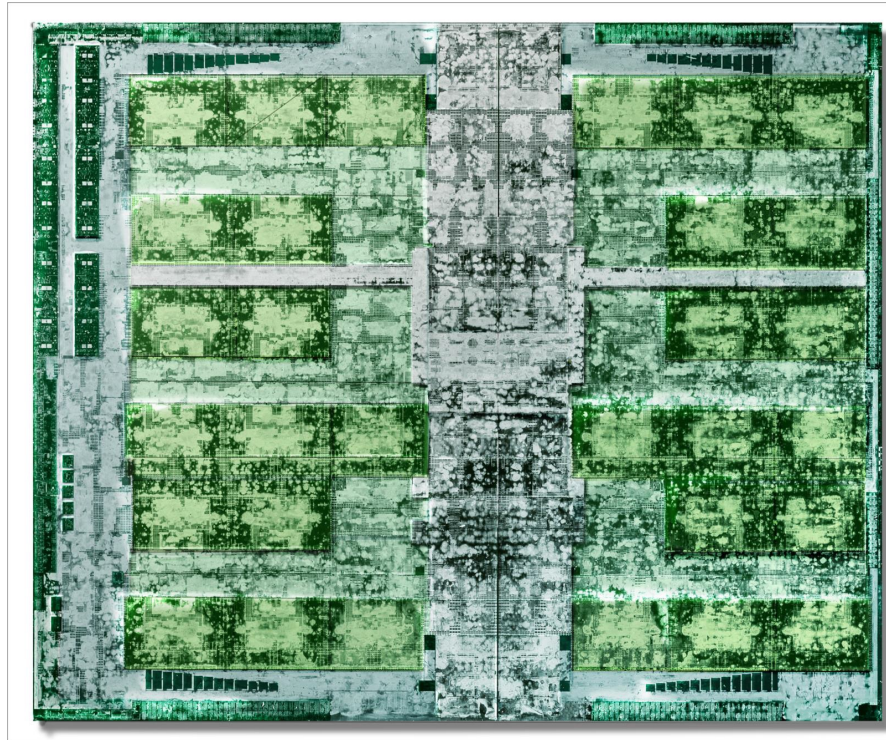
- Update the kernel launch:

```
add<<<(N + BLOCKSIZE-1)/BLOCKSIZE, BLOCKSIZE >>>(d_a, d_b, d_c, N);
```

Why Bother with Threads and Blocks?

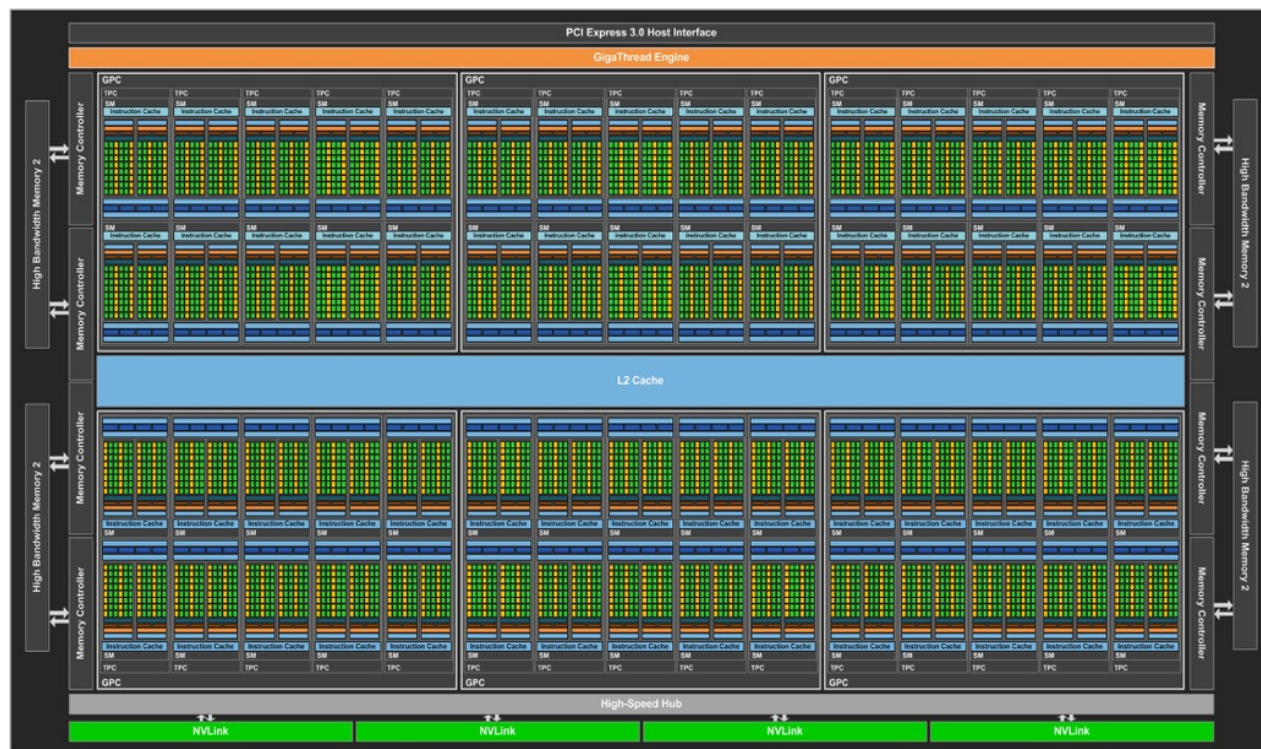
- Using both Threads and Blocks seems unnecessarily complicated
 - They add a level of complexity
 - What do we gain?
- Unlike parallel blocks, threads have mechanisms to:
 - Communicate
 - Synchronize

Pascal P100

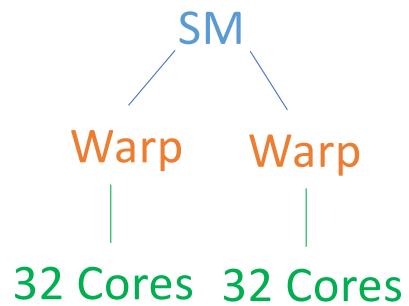


High Performance Computing

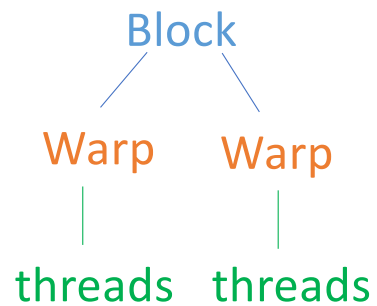
Pascal P100 diagram



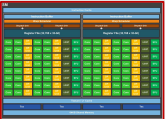
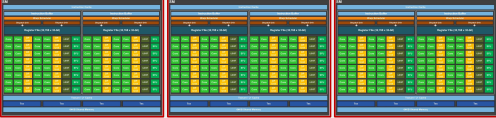
Streaming Multiprocessor (SM)



Streaming Multiprocessor (SM)



Terminology:

Concept	Hardware	Description
thread		Each Thread is executed by one core .
Warp		A Warp consist of 32 cores that share this register memory and operate simultaneously by construction.
Block		Each Block is assigned to an SM, consisting of Warps , and they share this 64KB memory.
Grid		The kernel is executed over a Grid of Blocks .

Recapitulation

- What have we learned?
 - Write and launch CUDA kernels
 - `__global__`, `<<<blocks, threads_per_block>>>`
 - `threadIdx.x`, `blockIdx.x`, `blockDim.x`
 - Manage GPU memory
 - `cudaMalloc()`, `cudaMemcpy()`, `cudaFree()`

Some extra topics:

- Host and Device coordination
- Reporting Errors
- 3D Blocks
- Shared memory
- Thread synchronization
- Device Management

Coordinating Host & Device

- Kernel launches are **asynchronous**
 - Control returns to the **Host** immediately
- **Host** needs to synchronize before consuming the results

`cudaMemcpy()`

Blocks the **Host** until the copy is complete.
Copy begins when all preceding kernel calls have been completed

`cudaMemcpyAsync()`

Asynchronous, does not block the **Host**

`cudaDeviceSynchronize()`

Blocks the **Host** until all preceding kernel calls have been completed

Reporting Errors

- All CUDA API calls return an error code (`cudaError_t`)
 - Error in the API call itself
 - OR
 - Error in an earlier asynchronous operation (e.g. kernel)

- Get the error code for the last error:

```
cudaError_t cudaGetLastError(void)
```

- Get a string to describe the error:

```
char *cudaGetErrorString(cudaError_t)
```

```
printf("%s\n", cudaGetErrorString(cudaGetLastError()));
```

3D Indexing

- A kernel is launched as a grid of blocks of threads

`blockIdx` and `threadIdx` are 3D

We showed only one dimension (x)

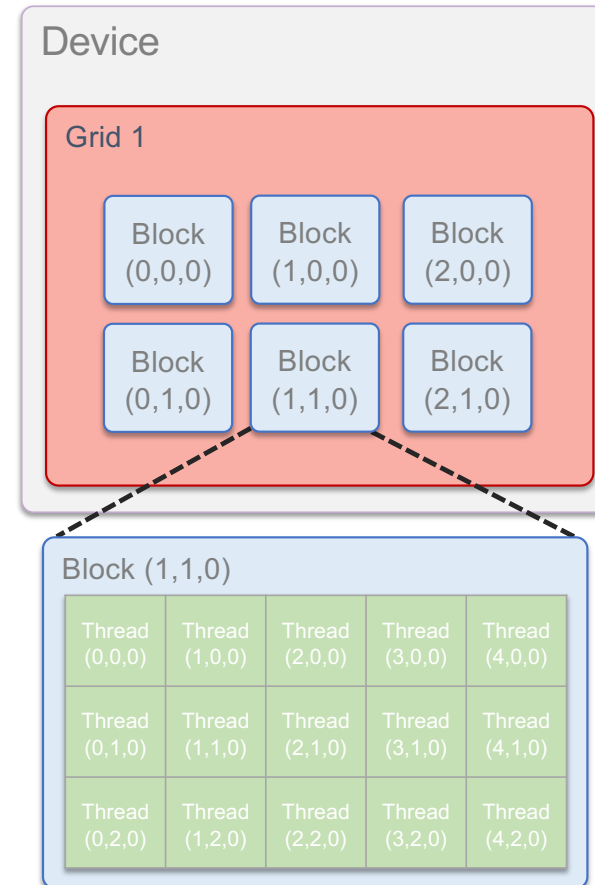
- Built-in variables:

`threadIdx` Thread indices (x,y,z)

`blockIdx` Block indices (x,y,z)

`blockDim` Threads per Block (x,y,z)

`gridDim` Blocks (x,y,z)



Sharing Data Between Threads

- Terminology: within a block, threads share data via **shared memory**
- Extremely fast on-chip memory, user-managed
- Declare using **__shared__**, allocated per block
- Data is not visible to threads in other blocks

__syncthreads()

```
void __syncthreads();
```

- Synchronizes all threads within a block
 - Used to prevent race conditions hazards
- All threads must reach the barrier
 - In conditional code, the condition must be uniform across the block

Device Management

- Application can query and select GPUs

```
cudaGetDeviceCount(int *count)
cudaSetDevice(int device)
cudaGetDevice(int *device)
cudaGetDeviceProperties(cudaDeviceProp *prop, int
device)
```

- Multiple host threads can share a device
- A single host thread can manage multiple devices
`cudaSetDevice(i)` to select current device